

中国矿业大学计算机学院实验报告

课程名称	并行算法与 GPU 编程	实验名称	实验二 GPU 多核编程
班级	大数据一班	姓名	王志豪
		学号	08232337
实验日期	2026.4.26		
实验报告要求：1. 实验目的 2. 实验内容（题目描述，算法描述，运行截图，调试情况） 3. 实验体会			
一、实验目的 掌握 CUDA C/CUDA C++，基于多核 CPU 和 GPU 的异构 CUDA 编程模型，CUDA 的编程优化，掌握基于 CUDA 并行算法的应用程序设计。 二、实验内容 2.1 题目描述 卷积运算是图像处理、计算机视觉以及深度学习中的核心计算操作，其计算过程具有高度的数据并行性。传统基于 CPU 的串行实现难以满足大规模数据处理的性能需求，而基于 GPU 的 CUDA 并行计算模型能够显著提升卷积运算的执行效率。 本实验要求基于 CUDA C/C++编程模型，设计并实现一个并行卷积算法。通过对比 CPU 串行实现与 GPU 并行实现的性能差异，理解 CUDA 线程层次结构（grid、block、thread）及其调度机制，掌握 CUDA 内存模型（全局内存、共享内存等）的使用方法，并在此基础上对卷积算法进行一定程度的性能优化。 具体要求如下： 1. 算法实现 - 实现二维卷积运算（可用于图像边缘检测、模糊等典型应用） - 编写 CPU 串行版本作为基准程序 - 使用 CUDA 实现 GPU 并行版本 2. 并行设计 - 合理划分线程块（block）与线程（thread） - 将卷积计算映射到 GPU 线程上，实现数据并行 3. 性能优化 - 尝试使用共享内存（shared memory）优化访存性能 - 分析不同线程块大小对性能的影响 - 减少全局内存访问次数，提高内存访问效率 4. 结果分析 - 对比 CPU 与 GPU 版本的运行时间 - 计算加速比（Speedup） - 分析性能瓶颈及优化效果			

2.2 算法描述

A. 问题定义

给定一个二维输入矩阵 $I \in \mathbb{Z}^{H \times W}$ 和一个二维卷积核 $K \in \mathbb{Z}^{K_h \times K_w}$ (通常 K_h 、 K_w 为奇数, 且 $K_h \leq H$, $K_w \leq W$)。

采用 VALID 卷积 (无填充, 步长 = 1), 输出矩阵 $O \in \mathbb{Z}^{H_o \times W_o}$ 定义为:

$$H_o = H - K_h + 1, W_o = W - K_w + 1,$$

且对于每个输出位置 (y, x) ($0 \leq y < H_o$, $0 \leq x < W_o$):

$$O[y, x] = \sum_{k_y=0}^{K_h-1} \sum_{k_x=0}^{K_w-1} I[y + k_y, x + k_x] \times K[k_y, k_x].$$

本算法使用 CUDA 并行计算所有输出点, 每个线程负责计算一个独立的输出点。

B. 并行策略与线程映射

- **全局网格划分:** 采用二维 CUDA 网格, 其中
 - $\text{grid.x} = \text{ceil}(W_o / \text{TILE_X})$,
 - $\text{grid.y} = \text{ceil}(H_o / \text{TILE_Y})$,
 - 每个线程块包含 $\text{TILE_X} \times \text{TILE_Y}$ 个线程 (本实现取 $\text{TILE_X} = \text{TILE_Y} = 16$)。
- **线程到输出坐标的映射:**

对于块索引 $(\text{blockIdx.x}, \text{blockIdx.y})$ 和块内线程索引 $(\text{threadIdx.x}, \text{threadIdx.y})$, 该线程负责的输出点坐标为:

$$\begin{aligned} \text{out}_x &= \text{blockIdx.x} \times \text{TILE_X} + \text{threadIdx.x}, \\ \text{out}_y &= \text{blockIdx.y} \times \text{TILE_Y} + \text{threadIdx.y}. \end{aligned}$$
- **有效性检查:** 若 $\text{out}_x \geq W_o$ 或 $\text{out}_y \geq H_o$, 该线程直接返回 (不执行任何计算)。

C. 单个线程的计算过程

每个有效线程执行以下步骤:

1. **初始化累加器:** 设置局部变量 $\text{sum} = 0$ 。
2. **遍历卷积核窗口:** 双重循环 $\text{ky} = 0..K_h-1$, $\text{kx} = 0..K_w-1$:
 - 计算输入矩阵中的对应像素坐标:
$$\begin{aligned} \text{in_row} &= \text{out_y} + \text{ky}, \\ \text{in_col} &= \text{out_x} + \text{kx}. \end{aligned}$$
(注意: VALID 卷积保证 $\text{in_row} < H$, $\text{in_col} < W$, 无需额外的边界检查。)
 - 从全局内存中读取输入元素 $\text{input}[\text{in_row} * W + \text{in_col}]$ 。
 - 从全局内存中读取卷积核元素 $\text{kernel}[\text{ky} * K_w + \text{kx}]$ 。
 - 执行乘加: $\text{sum} += \text{input_val} * \text{kernel_val}$ 。
3. **写回结果:** 将累加值 sum 写入输出矩阵的对应位置:
$$\text{output}[\text{out_y} * W_o + \text{out_x}] = \text{sum}.$$

C. 核心代码展示

共享内存优化:

```

// ----- 共享内存优化并行卷积 -----
#define TILE_Y 16
#define TILE_X 16

__global__ void conv2d_shared(const DataType *input, const DataType
*kernel,
                                int rows, int cols, int kRows, int kCols,
                                DataType *output, int outRows, int outCols)
{
    // 动态共享内存，大小为 shm_h * shm_w
    extern __shared__ DataType shmem[];
    int tx = threadIdx.x;
    int ty = threadIdx.y;

    // 当前块负责的输出图块左上角坐标
    int base_x = blockIdx.x * TILE_X;
    int base_y = blockIdx.y * TILE_Y;

    // 当前线程负责的输出点坐标
    int out_x = base_x + tx;
    int out_y = base_y + ty;

    // 输入图块尺寸（包含卷积所需的 halos）
    int shm_h = TILE_Y + kRows - 1;
    int shm_w = TILE_X + kCols - 1;

    // ----- 阶段 1: 将当前输入图块加载到共享内存（所有线程协同） -----
    // 每个线程负责加载多个点，保证覆盖整个 shm_h x shm_w 区域
    for (int i = ty; i < shm_h; i += TILE_Y)
    {
        for (int j = tx; j < shm_w; j += TILE_X)
        {
            int in_y = base_y + i;
            int in_x = base_x + j;
            if (in_y >= 0 && in_y < rows && in_x >= 0 && in_x < cols)
            {
                shmem[i * shm_w + j] = input[in_y * cols + in_x];
            }
            else
            {
                shmem[i * shm_w + j] = 0; // 边界外填充 0（VALID 卷积实际不需
要，但安全）
            }
        }
    }
}

```

```

    }
    __syncthreads();

    // ----- 阶段 2: 卷积计算（仅当输出点有效时） -----
    if (out_x < outCols && out_y < outRows)
    {
        int sum = 0;
        for (int ky = 0; ky < kRows; ++ky)
        {
            for (int kx = 0; kx < kCols; ++kx)
            {
                // 当前输出点对应的输入窗口在共享内存中的偏移就是 (ty + ky, tx +
                kx)

                DataType in_val = shmem[(ty + ky) * shm_w + (tx + kx)];
                DataType ker_val = kernel[ky * kCols + kx];
                sum += in_val * ker_val;
            }
        }
        output[out_y * outCols + out_x] = sum;
    }
}

```

朴素的并行卷积算法:

```

// ----- 朴素并行卷积（全局内存直接访问） -----
__global__ void conv2d_naive(const DataType *input, const DataType *kernel,
                             int rows, int cols, int kRows, int kCols,
                             DataType *output, int outRows, int outCols)
{
    int out_x = blockIdx.x * blockDim.x + threadIdx.x;
    int out_y = blockIdx.y * blockDim.y + threadIdx.y;
    if (out_x >= outCols || out_y >= outRows)
        return;

    int sum = 0;
    for (int ky = 0; ky < kRows; ++ky)
    {
        for (int kx = 0; kx < kCols; ++kx)
        {
            int in_row = out_y + ky;
            int in_col = out_x + kx;
            sum += input[in_row * cols + in_col] *
                    kernel[ky * kCols + kx];
        }
    }
    output[out_y * outCols + out_x] = sum;
}

```

```
}
```

2.3 运行截图

```
Input: 1024x1024, Kernel: 3x3, Output: 1022x1022
CPU time: 36.867 ms
GPU kernel time: 23.944 ms
Verification: PASS
Speedup (CPU/GPU): 1.54
```

由运行结果可以得出：

- 输入矩阵： 1024×1024 （约 1M 像素）
- 卷积核： 3×3
- 输出矩阵： 1022×1022 （约 1.04M 像素）
- CPU 串行时间：36.867 ms
- GPU 朴素内核时间：23.944 ms
- 加速比： $36.867 / 23.944 \approx 1.54$ 倍

但是这个加速比是远低于我们的预期，往往基于 CUDA 的并行卷积算法相对于串行的算法来说能够实现 $10 \sim 100$ 倍加速，但是这里提升极为有限，分析了原因可能如下：

全局内存冗余访问极度严重：

在朴素卷积实现中，每个输出像素需读取输入图像中一个 3×3 窗口（9 个像素）及对应卷积核（9 个值）。对于 1024×1024 图像，输出约 1.04M 像素，导致全局内存总读取量高达 9.4M 次（约 36 MB）。然而，相邻输出窗口存在严重重叠——每个输入像素平均被重复读取 9 次，这种冗余访问是性能低下的根本原因。

为此，我们利用共享内存优化了并行卷积算法，最终评测结果如下：

```
Input: 1024x1024, Kernel: 3x3, Output: 1022x1022
CPU serial time: 28.564 ms
GPU naive time: 41.256 ms
Naive verification: PASS
GPU shared memory time: 0.139 ms
Shared verification: PASS

Speedup (CPU/Naive): 0.69
Speedup (CPU/Shared): 205.10
Speedup (Naive/Shared): 296.24
```

可以看到，此次评测的结果，甚至朴素的并行算法甚至要比 CPU 串行的速度还要慢，原因不仅包括了上述的全局内存冗余访问的问题，还有可能是：CPU 虽然单核性能低于 GPU，但拥有多级缓存（L1/L2/L3）自动重用重叠的输入数据。当卷积核较小（ 3×3 ），CPU 的缓存命中率极高，实际访问主存的次数远少于 GPU 朴素版本，并且小规模数据量

计算时，多线程的创建对运行时间的影响甚至要大于串行算法本身的局限，综合以上的问题，朴素的并行算法的速度才会慢于 CPU 的串行算法速度。

2.4 调试情况

Shared verification: FAIL

在实验过程中，出现了共享内存优化后的结果与 CPU 版本和朴素并行版本的结果不同的错误，分析了源码后发现错误原因如下：

1. **共享内存加载的基地址错误：**原代码中每个线程用自己的 `out_x/out_y` 作为输入窗口左上角去加载共享内存，导致同一块内不同线程加载了不同的输入区域，共享内存数据混乱。
2. **边界线程提前退出：**输出超出范围的线程直接 `return`，导致共享内存未被完全填充，影响块内其他有效线程的计算。

修正的主要内容如下：

1. **统一基地址**
所有线程使用相同的 `base_x = blockIdx.x * TILE_X, base_y = blockIdx.y * TILE_Y` 来加载共享内存，确保块内加载的是同一块输入区域。
2. **边界线程处理**
 - 通道越界 (`out_chan >= out_c`) 直接返回。
 - 输出空间域越界 (`out_x >= out_w || out_y >= out_h`) 的线程**仍然参与共享内存加载**，但不进行卷积计算和写回，避免污染共享内存。
3. **卷积计算索引修正**
当前线程的输出点 (`out_y, out_x`) 在共享内存中对应的偏移就是 (`ty, tx`)，因此卷积窗口的共享内存坐标为 (`ty+ky, tx+kx`)，正确对应输入点 (`base_y+ty+ky, base_x+tx+kx`)。

三、实验体会

通过本次并行卷积实验，我深刻体会到：在 GPU 上实现并行算法并非简单地将串行循环映射到大量线程就能获得高性能。朴素的全局内存卷积尽管逻辑简单、线程完全并行，但每个线程独立重复读取重叠的输入数据，导致算术强度极低，访存成为严重瓶颈，最终执行时间甚至比 CPU 串行版本还要慢 31%。这一反例让我认识到，忽视数据重用和内存访问模式，单纯增加并行度反而会因全局带宽饱和、缓存缺失等开销而适得其反。

共享内存优化版本的引入则完美诠释了 GPU 优化的核心思想——**通过数据复用将频繁访问的数据从全局内存搬到片上共享内存**。

以 16×16 线程块为例，每个块只需加载一次带 halo 的输入图块 (18×18)，块内所有线程即可从共享内存中快速读取所需数据，全局内存读次数减少了 7 倍以上，访存延迟也大幅降低。最终该版本相对 CPU 串行获得了 205 倍的加速，相对朴素 GPU 更是提升了 296 倍。这让我明白，衡量并行算法的优劣不只是看线程数量，更要关注内存层次结构的使用效率；只有让计算和访存达到平衡，才能真正释放 GPU 的潜力。本次实验对我今后设计并行算法、优化 CUDA 程序具有重要的指导意义。

